TECHNICAL PROJECT DOCUMENTATION

ATLAS

Enterprise AI

A multi-tenant Retrieval-Augmented Generation knowledge assistant

Architecture | Workflows | Modules | API | Data | Security | Operations

DOCUMENT VERSION	PREPARED	PROJECT
1.0	November 2026	ATLAS Enterprise AI

Prepared for developers, reviewers, maintainers and technical stakeholders.
Verify endpoints, revisions, test results and environment settings against the repository before publishing as a live record.

Document information

Item	Value
Document version	1.0
Prepared	November 2026
Project	ATLAS Enterprise AI
Primary purpose	Explain the system, its implementation, and operating procedures
Source basis	User-provided ATLAS project notes and technical content

Important status note
This document describes the architecture and implementation details stated in the supplied project material. Before publishing it as a live production record, verify the current AWS endpoint, deployed revision, test results, exact dependency versions, and environment settings against the repository and cloud accounts.

Document purpose and how to use it

This document explains ATLAS from the user journey down to the implementation modules. It is written for developers, reviewers, maintainers, and technical stakeholders who need to understand how the system works and how to operate it.

- Start with **Sections 1–4** to understand the problem, solution, and architecture.
- Use **Sections 5–8** when navigating the repository or following a request through the code.
- Use **Sections 9–12** for data design, security, testing, and deployment.
- Use **Sections 13–16** for troubleshooting, limitations, planned improvements, and operational learning.

Reading convention
Commands and paths are shown in code formatting. Example IDs, tenant names, API payloads, and URLs are illustrative unless explicitly labelled as verified values.

Contents

1. Executive summary . . . 4
2. Product scope and user journey . . . 5
3. Technology stack . . . 6
4. System architecture . . . 7
5. Repository structure . . . 8
6. End-to-end workflows . . . 9
7. Application modules . . . 10
8. REST API reference . . . 12
9. Database design . . . 14
10. Security and governance . . . 15
11. Testing and quality assurance . . . 16
12. Deployment guide . . . 17
13. Operations runbook . . . 19
14. Known limitations . . . 20
15. Recommended improvement roadmap . . . 21
16. Engineering lessons . . . 22
17. Glossary . . . 23
Appendix A. Environment configuration . . . 23
Appendix B. Useful SQL queries . . . 24
Appendix C. Source-content validation notes . . . 25

1. Executive summary

1.1 What ATLAS is

ATLAS is a multi-tenant Retrieval-Augmented Generation (RAG) knowledge assistant. It allows an organization to store internal documents, ask questions in natural language, and receive answers based on retrieved document evidence with source references.

1.2 The problem

Enterprise information is often spread across PDFs, Word documents, text files, and internal knowledge sources. Staff must search and read these files manually. A general-purpose language model may not know private company information and can produce unsupported answers.

1.3 The solution

- Document ingestion for PDF, DOCX, TXT, and Markdown files.
- Semantic retrieval using vector embeddings and Pinecone.
- Answer generation using an OpenAI language model and retrieved evidence.
- Source metadata returned with answers to support verification.
- Tenant-scoped retrieval and database queries.
- Conversation memory, input guardrails, audit events, and approval workflows.

1.4 Intended users

User role	Typical activity
Employee / knowledge user	Upload permitted documents and ask questions about organizational knowledge.
Reviewer / administrator	Review sensitive requests and approve or reject pending approvals.
Developer / operator	Configure integrations, run tests, deploy the service, and investigate failures.

1.5 Scope boundaries

Included in the described version	Not included or identified as incomplete
Document upload and ingestion	User authentication and verified identity
RAG question answering with sources	Streaming responses
Tenant filters in retrieval and data queries	A complete document deletion workflow in the UI
Conversation history persistence	Native mobile application
Prompt-injection detection and HITL flow	Fine-tuned language or embedding models
Audit and operational event recording	A fully implemented RAG evaluation dashboard

2. Product scope and user journey

2.1 Upload a document

A user opens the Documents area, selects a supported file, supplies a tenant identifier and owner identifier, and submits the upload.

- The API validates the upload and stores the original file in S3 using a tenant-scoped object key.
- The API creates a document metadata record in PostgreSQL with a pending status.
- A background ingestion task is started.
- The UI can poll the document status until processing completes.

2.2 Process the document

The ingestion worker downloads the file, extracts text, splits the text into overlapping chunks, creates an embedding for each chunk, and stores the vectors in Pinecone with metadata.

Status	Meaning
pending	The upload has been accepted and is waiting for processing.
processing	The ingestion worker is processing the file.
ready	Text was processed and vector records were stored successfully.
failed	The ingestion process encountered an error.
retired	A document is no longer considered active, where this status is used.

2.3 Ask a question

- The API validates the question and identifiers.
- Guardrails check for invalid input and known prompt-injection patterns.
- The question is embedded using the configured embedding model.
- Pinecone retrieves the most relevant chunks, filtered by tenant ID.
- Retrieved text is sanitized and assembled as numbered evidence.
- Recent conversation turns are loaded from PostgreSQL.
- The configured OpenAI model generates an answer using the evidence and conversation context.
- The API returns the answer and source metadata, then stores the conversation turns.

2.4 Review a sensitive request

When safe mode is enabled and a question matches a configured sensitive-topic trigger, the workflow creates a pending approval instead of sending the question to answer generation. A reviewer can approve or reject the request. The described workflow records the reviewer and review timestamp.

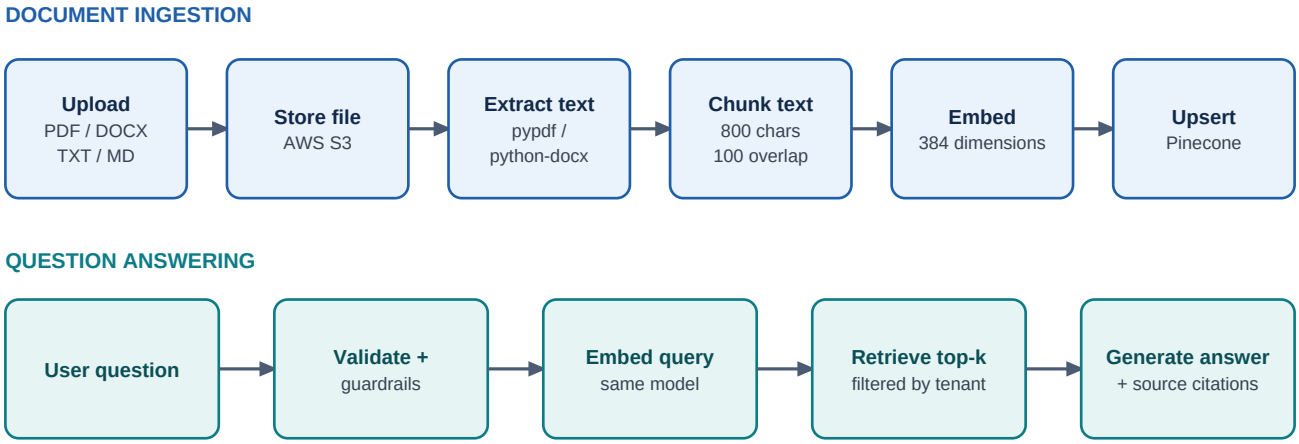


Figure 1. Simplified ingestion and question-answering pipelines.

3. Technology stack

Layer	Technology in supplied design	Role in ATLAS
API	FastAPI + Uvicorn	Exposes HTTP endpoints and serves the web application.
Validation / settings	Pydantic + pydantic-settings	Validates API payloads and loads application configuration.
Frontend	HTML, CSS, vanilla JavaScript	Provides Chat, Documents, Approvals, and Metrics views.
Relational database	PostgreSQL on Neon	Stores document metadata, chat history, approvals, feedback, and events.
Vector database	Pinecone	Stores and searches document-chunk embeddings.
Language model	OpenAI API	Generates natural-language answers from retrieved evidence.
Embedding model	BAAI/bge-small-en-v1.5 via sentence-transformers	Converts text and questions into 384-dimensional vectors.
Object storage	AWS S3	Stores original uploaded documents.
Cloud runtime	Docker + AWS ECS Fargate + ALB + ECR	Packages and hosts the API behind a load balancer.
Testing	pytest	Runs unit, integration, and end-to-end checks.

3.1 Why the components are separated

PostgreSQL stores structured records and relationships; Pinecone is optimized for similarity search over vectors; S3 stores original files; and the language model generates responses. Keeping these responsibilities separate makes each service easier to configure, scale, and troubleshoot.

Configuration must be authoritative
The supplied notes contain different model and region values in different sections. Treat the current project-root **.env**, **app/config.py**, Pinecone index settings, and AWS task definition as the source of truth. Do not copy a sample value into production without checking it.

4. System architecture

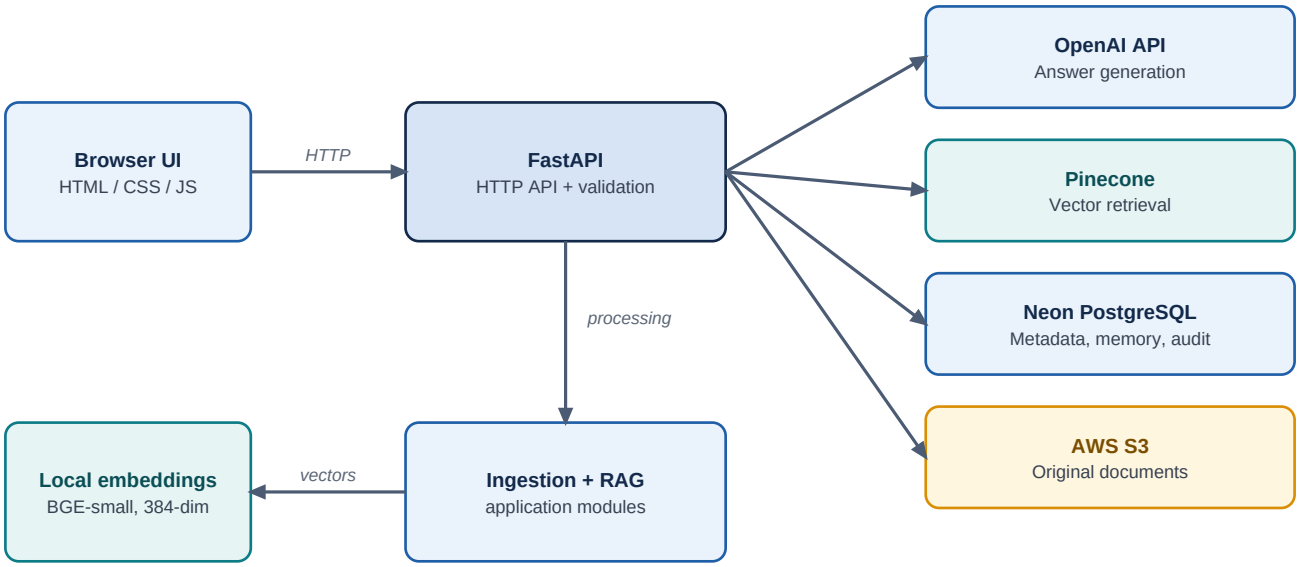


Figure 2. Logical architecture of the application and external services.

4.1 Presentation layer

The browser-based interface is served by FastAPI. The described frontend uses standard HTML, CSS, and JavaScript and does not require a separate frontend build process.

4.2 Application layer

FastAPI receives requests, validates payloads, coordinates RAG and ingestion modules, and returns structured responses. The application also serves static assets and manages startup and shutdown lifecycle tasks.

4.3 Retrieval and generation layer

The retrieval layer embeds a question and searches Pinecone. The generation layer passes the question, retrieved evidence, and selected conversation history to the configured language model.

4.4 Data and integration layer

Neon PostgreSQL stores relational application data, Pinecone stores vectors, and S3 stores document bytes. These are separate services and should be monitored independently.

4.5 Concurrency and background work

- The supplied design runs Uvicorn with a single application process.
- Database operations use psycopg and a connection pool; the notes specify a maximum pool size of four.
- FastAPI BackgroundTasks performs ingestion in the application process.
- The embedding model is cached so it can be reused rather than loaded for every request.

Scaling consideration
In-process background tasks can be interrupted if the application process stops. A durable queue such as SQS or Celery with a broker is a recommended future improvement for production workloads.

5. Repository structure

```
ATLAS-Enterprise-AI/
|-- app/
|   |-- main.py            # FastAPI routes and application lifecycle
|   |-- config.py          # Environment settings and validation
|   |-- db.py              # PostgreSQL connection pool and queries
|   |-- observability.py   # Tracing and operational events
|   |-- guardrails.py      # Input checks and evidence sanitization
|   |-- aws.py             # S3 client and file operations
|   |-- vector.py          # Pinecone vector operations
|   |-- memory.py          # Conversation persistence
|   |-- rag.py             # Linear RAG pipeline
|   |-- graph.py           # Multi-step RAG and HITL orchestration
|   |-- approvals.py       # Approval workflow
|   |-- ingest/
|   |   |-- extractor.py   # PDF, DOCX, TXT extraction
|   |   `-- chunker.py     # Text splitting
|   |-- templates/
|   |   `-- index.html     # Main web page
|   `-- static/
|       |-- css/styles.css # UI styling
|       `-- js/app.js      # Browser interactions
|-- worker/
|   `-- ingest.py          # Document ingestion orchestration
|-- infra/
|   `-- schema.sql         # PostgreSQL schema
|-- scripts/
|   |-- init_db.py         # Apply database schema
|   |-- smoke_test.py      # Check external integrations
|   `-- check_events.py    # Inspect event records
|-- test/                  # Unit, integration, and E2E tests
|-- docs/                  # Project and deployment documentation
|-- Dockerfile
|-- requirements.txt
|-- pytest.ini
|-- conftest.py
|-- .env.example
|-- .gitignore
`-- README.md
```

Path	Responsibility
app/main.py	HTTP routes, request/response models, frontend serving, application lifecycle.
app/config.py	Reads environment variables and validates configuration.
app/db.py	Manages PostgreSQL connections and reusable database operations.
app/observability.py	Records execution events, duration, and errors.
app/guardrails.py	Validates identifiers and questions; detects suspicious input and sanitizes evidence.
app/aws.py	Uploads, downloads, checks, and deletes S3 objects; creates presigned URLs.
app/vector.py	Upserts, searches, and deletes Pinecone vectors with tenant filters.
app/memory.py	Stores and retrieves conversation turns.

Path	Responsibility
app/rag.py	Runs the direct validate-retrieve-generate-persist sequence.
app/graph.py	Coordinates multi-step retrieval, query rewriting, and HITL checks.
app/approvals.py	Creates, lists, retrieves, decides, and cancels approval requests.
app/ingest/extractor.py	Extracts text from supported file formats.
app/ingest/chunker.py	Splits extracted text into overlapping pieces.
worker/ingest.py	Coordinates the complete document ingestion lifecycle.
infra/schema.sql	Defines relational tables, constraints, and indexes.

Repository accuracy
The tree above is based on the supplied project notes. Confirm every listed file exists in the current checkout before treating this as an exact inventory. The documentation does not assume that an item in the notes is present in the latest repository revision.

6. End-to-end workflows

6.1 Document ingestion sequence

- Receive the upload request and validate its metadata and file size.
- Write the original bytes to S3 under a tenant-scoped key.
- Insert or update the document record in PostgreSQL.
- Mark the record as processing when the worker starts.
- Download the original file from S3.
- Extract text according to file type: PDF, DOCX, TXT, or Markdown.
- Split the extracted text into chunks (the supplied default is approximately 800 characters with 100 characters of overlap).
- Generate one 384-dimensional embedding per chunk using the configured embedding model.
- Create Pinecone records containing tenant ID, document ID, chunk index, version, and text metadata.
- Upsert records to Pinecone and mark the document ready.
- Write audit information. If processing fails, mark the document failed and record the error.

6.2 RAG question-answering sequence

- Receive question, tenant ID, conversation ID, top_k, and safe-mode settings.
- Validate the tenant and conversation identifiers.
- Check question length and invalid characters; flag known injection patterns.
- Check the HITL gate when safe mode is enabled.
- Generate a query embedding using the same embedding model used for documents.
- Search Pinecone using top_k and a mandatory tenant metadata filter.
- Sanitize the retrieved text and format it as numbered evidence.
- Load recent messages for the same tenant and conversation.
- Call the configured LLM with a system instruction that treats retrieved passages as untrusted data.

- Return the answer and source metadata; persist the user and assistant messages.

6.3 Chunking example

Chunking divides a long document into smaller pieces so that each piece can be embedded and retrieved independently. Overlap carries some text from one chunk into the next so that a sentence or idea near a boundary is less likely to lose its context.

```
Example settings from the supplied notes:
chunk_size = 800 characters
overlap    = 100 characters

Chunk 1: characters 0-799
Chunk 2: characters 700-1499
Chunk 3: characters 1400-2199
```

6.4 Why the embedding dimensions matter

The supplied configuration uses BAAI/bge-small-en-v1.5 and a vector size of 384. Pinecone index dimensions must match the vectors sent by the application. If the index is configured for a different dimension, vector upsert or query operations will fail.

6.5 HITL workflow

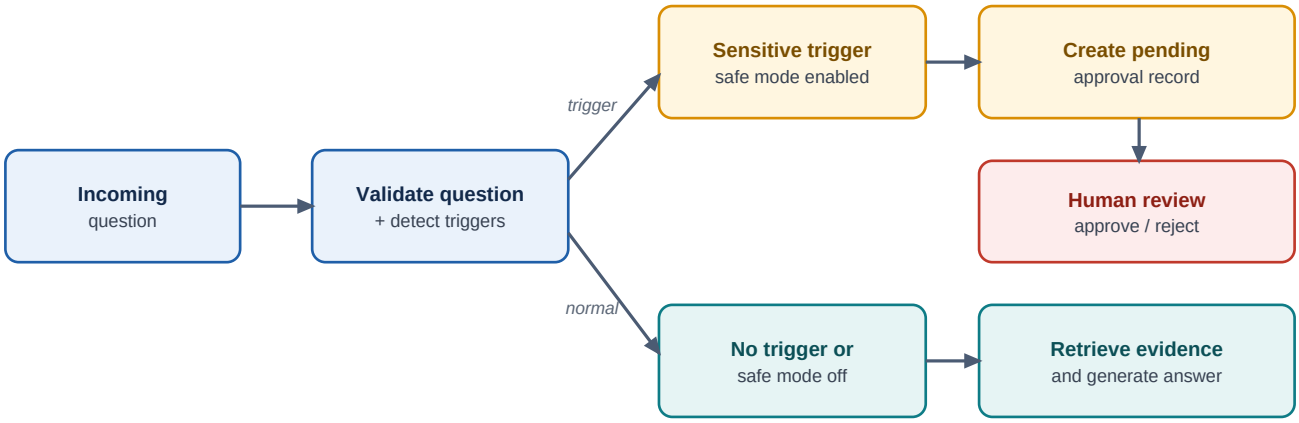


Figure 3. Simplified approval decision flow. The actual trigger words and routing rules are configured in the application.

- **Safe mode enabled + sensitive trigger:** create a pending approval and skip answer generation.
- **Reviewer decision:** mark the request approved or rejected and record reviewer information.
- **Safe mode disabled:** the supplied notes describe normal RAG processing even when a trigger word is present. This behavior should be reviewed before production use.

7. Application modules

7.1 Configuration — app/config.py

Centralizes environment-based settings. The supplied design uses Pydantic Settings and cached settings construction. Validators check related values such as embedding dimensions, AWS credential pairs, and Pinecone host format.

7.2 Database — app/db.py

Creates and manages a psycopg connection pool. A bounded pool reduces connection churn and helps protect a small database plan from excessive concurrent connections. The supplied notes specify min_size=1 and max_size=4.

7.3 Observability — app/observability.py

Provides tracing decorators and context managers. Events can include event type, duration, success or failure, tenant, actor, trace ID, and structured detail. Observability should not cause the main request to fail if event recording is unavailable.

7.4 Guardrails — app/guardrails.py

Validates questions and identifiers, detects known prompt-injection patterns, and sanitizes retrieved evidence. Pattern detection is a signal, not a complete security boundary.

7.5 AWS storage — app/aws.py

Wraps boto3 operations for S3. Typical operations include upload_bytes, download_bytes, delete_object, exists, and presigned_url. In AWS, prefer a task role over long-lived access keys.

7.6 Vector operations — app/vector.py

Wraps Pinecone upsert, query, delete, tenant cleanup, and index statistics. Each retrieval must include a tenant filter; the caller must not be able to omit it accidentally.

7.7 Conversation memory — app/memory.py

Stores user and assistant messages and retrieves recent turns using tenant_id and conversation_id. Both identifiers should be included in every query to prevent cross-tenant or cross-conversation access.

7.8 Linear RAG — app/rag.py

Implements a single-pass RAG sequence: validate, embed, retrieve, format evidence, load history, generate, persist, and return structured output.

7.9 Graph workflow — app/graph.py

Coordinates multiple steps such as validation, HITL checks, retrieval, optional query rewriting, generation, and persistence. It also returns a step trace to help explain which nodes ran.

7.10 Approval workflow — app/approvals.py

Persists approval requests and supports listing, fetching, approving, rejecting, and cancelling them. A decision should be final unless an explicit reopening policy is implemented.

7.11 Text extraction — app/ingest/extractor.py

Extracts text from PDF with pypdf, DOCX with python-docx, and text-based files by decoding bytes. Scanned image-only PDFs require OCR and are outside the described extractor.

7.12 Chunking — app/ingest/chunker.py

Splits extracted text into manageable overlapping chunks. The supplied default is 800 characters with 100 characters of overlap, with attempts to preserve paragraph or sentence boundaries.

7.13 Ingestion worker — worker/ingest.py

Coordinates database lookup, S3 download, extraction, chunking, embedding, Pinecone upsert, status updates, and audit events. On failure it marks the document failed and records an error.

7.14 API application — app/main.py

Defines request/response models and routes, serves the UI, mounts static files, and manages startup/shutdown resources such as the database pool.

7.15 Frontend — app/templates and app/static

Provides the browser interface. The supplied notes describe Chat, Documents, Approvals, and Metrics tabs. JavaScript cache-busting query strings may be used to avoid stale browser assets.

8. REST API reference

The following routes are listed in the supplied notes. Confirm the current route definitions in **app/main.py** and the automatically generated OpenAPI document before publishing an API contract.

Method	Path	Purpose
GET	/	Serve the main web interface.
GET	/health	Check application and PostgreSQL health.
POST	/upload	Accept a document upload and start ingestion.
POST	/query	Run a RAG question or safe-mode workflow.
GET	/documents/{tenant_id}	List documents for a tenant; may accept status filtering.
GET	/documents/status/{document_id}	Check ingestion status for a document.
POST	/feedback	Store user feedback for an answer.
GET	/api/approvals	List pending approvals for a tenant.
GET	/api/approvals/all	List approvals across statuses for a tenant.
GET	/api/approvals/{approval_id}	Retrieve one approval record.
POST	/api/approvals/{approval_id}/decision	Approve or reject an approval request.
POST	/api/approvals/{approval_id}/cancel	Cancel an approval request.

8.1 Health check

```
GET /health
```

Example healthy response:

```
{
  "status": "ok",
  "components": {"postgres": "ok"}
}
```

Example degraded response:

```
{
  "status": "degraded",
  "components": {"postgres": "down"}
}
```

8.2 Upload document

```
POST /upload
Content-Type: multipart/form-data

file      = <file bytes>
tenant_id = demo
owner_sub = alice
```

Example accepted response (HTTP 202):

```
{
  "document_id": "<uuid>",
  "s3_key": "demo/2026/<uuid>-filename.txt",
  "status": "pending",
  "message": "Upload accepted. Processing in background."
}
```

8.3 Query

```
POST /query
Content-Type: application/json

{
  "question": "What is the refund window?",
  "tenant_id": "demo",
  "conversation_id": "550e8400-e29b-41d4-a716-446655440000",
  "top_k": 5,
  "safe_mode": false,
  "actor": "demo-user"
}
```

A normal response contains the generated answer, source records, conversation ID, suspicious-input indicators, and workflow metadata. A safe-mode response may instead contain **needs_approval=true**, an approval ID, and no generated answer.

8.4 List documents and poll status

```
GET /documents/demo?status=ready
GET /documents/status/<document_id>
```

8.5 Approval operations

```
GET  /api/approvals?tenant_id=demo
GET  /api/approvals/all?tenant_id=demo&limit=20
GET  /api/approvals/<approval_id>

POST /api/approvals/<approval_id>/decision
{
  "reviewer": "admin",
  "approve": true,
  "note": "Reviewed"
}

POST /api/approvals/<approval_id>/cancel?actor=alice
```

8.6 Interactive API documentation

FastAPI commonly exposes an OpenAPI schema and interactive Swagger UI. If enabled in the current application configuration, use **/docs** on the running API to inspect the actual request models, response models, and route definitions. The live API schema is authoritative when it differs from this reference.

9. Database design

PostgreSQL stores structured records and operational history. Pinecone stores vector embeddings and chunk metadata; S3 stores the original uploaded files.

Table	Purpose stated in source
documents	Document metadata, tenant ownership, storage key, status, and version.
document_versions	Version history for documents.
feedback	User ratings and notes.
audit_events	Security and business activity history.
approvals	Human review requests and decisions.
chat_messages	Conversation turns.
observability_events	Operational traces and structured event details.

9.1 Key table concepts

documents

The documents record associates a file with a tenant, title, owner, S3 key, processing status, version, and timestamps. The supplied example also includes allowed_groups for access-control metadata.

```
documents (illustrative columns)
id, tenant_id, title, owner_sub, allowed_groups,
s3_key, status, current_version, created_at, updated_at
```

audit_events

Stores who performed an action, which tenant it affected, what action occurred, structured details, and when it happened.

```
audit_events: id, actor, tenant_id, action, detail JSONB, created_at
```

approvals

Stores the request, tenant, requester, status, reviewer, creation time, and review time. The supplied schema restricts status to pending, approved, rejected, or cancelled.

chat_messages

Stores a role and message content under a tenant and conversation ID. The role is restricted to user or assistant.

observability_events

Stores event type, tenant, optional actor and trace ID, structured details, and timestamp.

9.2 Relationships and data ownership

- A tenant can own many documents, conversations, feedback records, approvals, and events.
- A conversation is identified by tenant_id plus conversation_id.
- A document's original bytes are held in S3; its searchable chunks are held in Pinecone; its descriptive metadata is held in PostgreSQL.
- Document versions and access-control group membership should be checked against the full infra/schema.sql before extending the data model.

Schema completeness
The source lists seven tables but provides full CREATE TABLE examples for only a subset. The complete, current **infra/schema.sql** file—not this summary—must be used to create or migrate the database.

10. Security and governance

10.1 Tenant isolation

Tenant isolation means that one organization must not retrieve or read another organization's information. The described design enforces this in vector retrieval and relational database queries.

```
# Pinecone query filter - illustrative
filter={"tenant_id": {"$eq": tenant_id}}

# PostgreSQL query pattern - illustrative
SELECT * FROM documents WHERE tenant_id = %s;
SELECT * FROM chat_messages
WHERE tenant_id = %s AND conversation_id = %s;
```

Security requirement
Tenant identity must come from a trusted authenticated identity in a production system. A tenant_id supplied freely by an unauthenticated client is not a secure authorization boundary.

10.2 Input validation

Control	Purpose described in source
Question length	Limit questions to 1–2,000 characters.
Control characters	Reject NUL bytes and remove disallowed control characters.
Tenant ID	Restrict to alphanumeric characters, hyphen, and underscore; 1–64 characters.
Conversation ID	Require a valid UUID.
Injection detection	Flag common prompt-injection patterns for handling and monitoring.
Evidence sanitization	Strip control characters and cap retrieved evidence length.

10.3 Prompt-injection handling

The supplied design uses pattern detection and a hardened system instruction telling the model to treat retrieved passages as untrusted data, not as instructions. This is defense in depth, not a guarantee that prompt injection is impossible.

10.4 Secrets and cloud permissions

- Keep real credentials in a local .env file or a managed secret store; never commit them.
- Keep .env.example limited to placeholder values.
- Use an AWS task role for ECS workloads instead of long-lived access keys where possible.
- Grant the application only the S3 and other AWS permissions it needs.
- Do not expose database URLs, API keys, access tokens, or sensitive document text in logs.

10.5 Human approval

The source describes sensitive-topic triggers such as compensation, legal matters, medical topics, credentials, termination, and destructive operations. The exact trigger list should be treated as configurable policy, not a substitute for authorization checks or legal/compliance review.

10.6 Important security limitation

> **Authentication is not implemented**
> The supplied notes explicitly identify the absence of authentication. If the current deployment still has no login, API key, or verified user identity, it must be treated as a demonstration environment—not as a secure multi-tenant production service. Tenant filters alone do not prove that a caller is entitled to use a given tenant ID.

11. Testing and quality assurance

11.1 Test categories

Category	What it verifies	Examples
Unit tests	Pure logic without requiring live services.	Configuration validation, guardrails, chunking, text extraction.
Integration tests	Communication with an external service or database.	Neon, S3, Pinecone, approval persistence.
End-to-end tests	A complete user or system workflow.	Upload, ingest, retrieve, answer; HTTP routes.

11.2 Commands

```
# Run the complete suite
pytest -v

# Run fast logic-focused tests
pytest test/test_guardrails.py test/test_chunker.py test/test_extractor.py -v

# Run one module
pytest test/test_vector.py -v

# Run one critical test
pytest test/test_vector.py::test_query_does_not_leak_across_tenants -v
```

11.3 Critical regression checks

Test described in source	Risk it is intended to prevent
test_query_does_not_leak_across_tenants	Cross-tenant vector retrieval.
test_get_recent_is_tenant_scoped	Cross-tenant conversation history access.
test_list_pending_is_tenant_scoped	Cross-tenant approval visibility.
test_hitl_trigger_returns_pending_approval	Sensitive request bypassing the approval gate.
test_detect_prompt_injection_catches_known_patterns	Known malicious input patterns going undetected.
test_validate_tenant_rejects_invalid	Unsafe tenant identifiers.

Test-count discrepancy
The supplied document claims 146/146 tests in one section, but the individual test counts listed in its inventory add up to 165. These figures cannot both describe the same test run. Run **pytest -q** in the current repository and replace all counts with the actual collected and passed totals. Do not publish the historical totals as verified results.

11.4 Before deployment

- Run the full test suite against the intended test environment.
- Confirm test fixtures clean up test records and objects.
- Verify the Pinecone index dimension matches the embedding output.
- Verify tenant-isolation and HITL regression tests pass.
- Check that no secrets are tracked by Git.
- Run an application health check and a complete upload/query smoke test.

12. Deployment guide

The supplied deployment design uses Docker, Amazon ECR, Amazon ECS Fargate, an Application Load Balancer, Neon PostgreSQL, Pinecone, OpenAI, and S3. The commands below are a deployment outline; verify account IDs, regions, ARNs, task roles, and security-group rules before execution.

12.1 Prerequisites

- Docker Desktop installed and running.
- AWS CLI configured with an appropriately restricted deployment identity.
- Access to the Neon database, Pinecone index, OpenAI API, and S3 bucket.
- A working .env for local testing and a secure method for supplying production secrets.

12.2 Build and test locally

```
docker build -t atlas .
docker run --rm -p 8000:8000 --env-file .env atlas

# In another terminal
curl http://localhost:8000/health
```

If port 8000 is already occupied, identify the existing container or process. Do not terminate an unrelated service without checking it first. A temporary alternative is to map another host port, for example **-p 8001:8000**.

12.3 Push the image to ECR (PowerShell)

```
$REGION   = "ap-southeast-2"
$REPO     = "atlas-enterprise-ai"
$ACCOUNT  = aws sts get-caller-identity --query Account --output text
$REGISTRY = "$ACCOUNT.dkr.ecr.$REGION.amazonaws.com"

aws ecr create-repository --repository-name $REPO --region $REGION

aws ecr get-login-password --region $REGION |
  docker login --username AWS --password-stdin $REGISTRY

docker tag atlas:latest "$REGISTRY/$REPO:latest"
docker push "$REGISTRY/$REPO:latest"
```

12.4 Create ECS resources

- Create an ECS cluster using the Fargate launch type.
- Create a task definition with the ECR image, container port 8000, CPU/memory sizing, log configuration, health check, and required environment settings.
- Use an ECS task execution role to pull the image and write logs; use a separate task role for application AWS permissions.
- Create an ECS service with the required subnets and security groups.
- Attach an Application Load Balancer listener and a target group that forwards to container port 8000.
- Configure the target group health check path as /health.
- Restrict inbound access to the load balancer and restrict application/database access to the minimum required network paths.
- Supply production secrets through an appropriate secret-management mechanism rather than embedding them in the image.

12.5 Database connectivity

Confirm that the deployed application can reach Neon over TLS. Avoid opening a database to the entire internet as a default workaround. Use the provider's supported network controls and the narrowest practical access policy.

12.6 Deployment verification

```
# Check the public or internal load-balancer endpoint
curl https://<your-domain-or-load-balancer>/health

# Inspect ECS service state
aws ecs describe-services \
  --cluster <cluster-name> \
  --services <service-name> \
  --region <region>
```

A successful deployment requires more than a running task: verify the target group is healthy, the health endpoint reports the database status correctly, and a controlled upload/query workflow succeeds.

12.7 Redeployment outline

```
docker build -t atlas .
docker tag atlas:latest <account>.dkr.ecr.<region>.amazonaws.com/atlas-enterprise-ai:latest
docker push <account>.dkr.ecr.<region>.amazonaws.com/atlas-enterprise-ai:latest

aws ecs update-service \
  --cluster <cluster-name> \
  --service <service-name> \
  --force-new-deployment \
  --region <region>
```

12.8 Cost and teardown

ECS Fargate and an Application Load Balancer can continue to incur charges while provisioned. Set the desired task count to zero when a demo environment is not needed, while understanding that the load balancer and other resources may still incur costs.

Destructive operations
Deleting an ECS service, cluster, load balancer, ECR repository, database, Pinecone index, or S3 bucket can be irreversible. Confirm resource identifiers and backups before running deletion commands.

13. Operations runbook

13.1 Health checks

```
# Local application
curl http://localhost:8000/health

# Running containers
docker ps
docker ps -a

# Application logs
docker logs <container-name> --tail 200

# ECS service summary
aws ecs describe-services --cluster <cluster> --services <service> --region <region>
```

13.2 Common issues

Symptom	Possible cause	First response
503 from load balancer	No healthy targets or application task is stopped.	Inspect ECS events, target health, and container logs.
Container exits at startup	Invalid environment value, missing secret, import error, or failed dependency.	Read docker logs; inspect the first traceback cause.
Database connection failure	Wrong DATABASE_URL, TLS issue, network rule, or provider outage.	Check the URL format without exposing credentials; test network access.
Pinecone dimension error	Index dimension differs from embedding output.	Confirm EMBED_DIM and the index dimension are both 384 for the described model.
Pinecone timeout	Index host/region mismatch, readiness delay, or network problem.	Check index host, cloud, region, API key, and readiness settings.
S3 AccessDenied	Missing IAM permission or incorrect bucket/region.	Review task role policy, bucket name, and AWS region.
Too many database connections	Pool size or total task count exceeds provider limits.	Inspect pool configuration and number of application replicas.
High memory use / OOM	Embedding model and framework memory requirements exceed task memory.	Review container memory metrics and right-size the task.
Old JavaScript still appears	Browser or proxy cache is serving a previous asset.	Use cache-busting asset versions and verify the deployed HTML.
Ingestion stuck in processing	Background task stopped or failed before status update.	Inspect application logs, document row, S3 object, and worker error.

13.3 Incident investigation order

- Check the health endpoint and determine whether the issue affects the API, database, or all services.
- Check container status and the latest application logs.
- Inspect ECS service events and target group health when running on AWS.
- Verify environment configuration and secret availability without printing secret values.
- Check PostgreSQL, Pinecone, and S3 connectivity separately.
- Reproduce with a small test document and a known question.
- Record the root cause and the corrective action in the project issue log.

14. Known limitations

Limitation in supplied notes	Impact	Recommended response
No authentication	A caller may be able to choose a tenant identifier; tenant filters alone do not authenticate users.	Implement verified identity, authorization, and tenant membership checks before real-user access.
HTTP-only deployment described	Traffic is not protected by TLS.	Use HTTPS with a managed certificate and redirect HTTP to HTTPS.

Limitation in supplied notes	Impact	Recommended response
No complete document deletion UI	Users cannot fully remove documents through the described interface.	Implement coordinated deletion from S3, Pinecone, and PostgreSQL with audit records.
Single task / no autoscaling	Limited capacity and a single runtime failure domain.	Add health-based scaling and multiple tasks after load testing.
No streaming	The user waits for the complete response.	Add Server-Sent Events or another streaming transport.
Scanned PDFs not OCR-processed	Image-only PDFs may produce no extracted text.	Add OCR such as Textract or Tesseract, with quality checks.
Conversation continuity limited	A page refresh may create a new conversation ID.	Persist conversation selection and provide a conversation history UI.
Feedback not used for optimization	Ratings are collected but do not automatically improve retrieval or prompts.	Add feedback analytics and a reviewed evaluation loop.
Single LLM provider	Provider outage or quota issues can interrupt generation.	Add provider abstraction and a tested fallback strategy.
In-process background tasks	Ingestion may be lost if the API process stops.	Move ingestion to a durable queue and separately managed worker.
No formal schema migration framework	Schema changes require manual care.	Adopt versioned migrations and rollback procedures.

14.1 Production-readiness position

Treat as a demonstrator or controlled pilot

Based on the limitations explicitly listed in the supplied material, the system should be treated as a demonstrator or controlled pilot until authentication, authorization, HTTPS, secret management, durable ingestion, and operational controls have been verified.

15. Recommended improvement roadmap

Priority	Improvement	Reason / outcome
P0 — Security	Authentication, tenant authorization, HTTPS, rate limiting	Protects real users and confidential enterprise data.
P0 — Reliability	Durable ingestion queue, retries, idempotency, dead-letter handling	Prevents document processing jobs from disappearing during restarts.
P1 — Data lifecycle	Document deletion and version cleanup	Supports retention, correction, and compliance requirements.
P1 — Retrieval quality	Hybrid search, reranking, query rewriting, retrieval evaluation	Improves relevance and answer grounding.
P1 — Observability	Centralized structured logs, metrics, traces, alerts	Reduces time to diagnose incidents.
P1 — Quality	RAG evaluation dataset and regression thresholds	Measures retrieval relevance, groundedness, and answer quality.

Priority	Improvement	Reason / outcome
P2 — User experience	Streaming responses and conversation history	Improves usability and continuity.
P2 — Scalability	Multiple tasks, autoscaling, worker separation, load testing	Supports higher concurrency and larger ingestion volumes.
P2 — Resilience	LLM provider abstraction and fallback	Reduces dependency on one model provider.
P3 — Advanced capabilities	OCR, multilingual retrieval, multi-hop RAG, SSO	Expands supported content and enterprise use cases.

15.1 Suggested implementation order

- Close the authentication and tenant-authorization gap.
- Enable HTTPS and harden network access.
- Move ingestion to a durable worker queue.
- Add document deletion and version lifecycle handling.
- Create a repeatable RAG evaluation dataset and baseline.
- Add retrieval improvements such as hybrid search and reranking.
- Add streaming, conversation history, and user-facing feedback analytics.
- Load-test, scale horizontally, and introduce provider fallback only after measuring failure modes.

16. Engineering lessons

16.1 Configuration and integrations

- Keep configuration centralized and validate related settings together.
- Embedding model dimensions must match the vector index configuration.
- Test cloud integrations with small smoke tests before running the full application.
- Use connection pooling and explicit application shutdown to avoid resource leaks.

16.2 RAG quality

- Chunk size and overlap influence retrieval quality and should be evaluated against representative documents.
- A relevance threshold is a product-quality setting: lower values retrieve more candidates but may add noise.
- Citations help users inspect evidence, but citation presence alone does not prove that an answer is correct.
- Prompt-injection detection is useful but cannot replace authorization, evidence boundaries, and output validation.

16.3 Deployment and operations

- A container image building successfully does not prove the application can start with production settings.
- Health checks, target-group configuration, task memory, and database connectivity must be validated together.
- Inspect logs before changing infrastructure; startup tracebacks often identify the first failing dependency.
- Keep documentation aligned with the actual repository and deployed revision.

17. Glossary

Term	Simple explanation
RAG	Retrieval-Augmented Generation: retrieve relevant private information, then ask a language model to answer using it.
LLM	Large Language Model: a model that understands and generates text.
Embedding	A list of numbers representing the meaning of text, used for similarity search.
Vector	A numerical representation such as a 384-dimensional embedding.
Vector database	A database designed to search vectors by similarity.
Chunk	A smaller section of a document created for retrieval.
Tenant	An organization or customer whose data must be kept separate.
HITL	Human-in-the-Loop: a person reviews a request before the system proceeds.
Guardrail	A validation or safety check around model inputs and outputs.
Prompt injection	Text that attempts to manipulate the model into ignoring its intended instructions.
Metadata	Structured information describing a document or vector, such as tenant ID or source file.
Reranking	A second-stage model or method that reorders retrieved results by relevance.
Hybrid search	A retrieval method combining semantic vector search with keyword search.
ACL	Access Control List: rules describing who may access a resource.
Audit event	A record of an important action for accountability and investigation.
Observability	The ability to understand system behavior through logs, metrics, and traces.
ECR	Amazon Elastic Container Registry: a service for storing container images.
ECS	Amazon Elastic Container Service: a service for running containers.
Fargate	A managed compute option for running containers without managing EC2 servers.
ALB	Application Load Balancer: routes incoming requests to application targets.
S3	Amazon Simple Storage Service: object storage for files.
Neon	A managed PostgreSQL database service.
Pinecone	A managed vector database service.
E2E test	An end-to-end test that checks a complete workflow across multiple components.

Appendix A. Environment configuration

The supplied notes refer to the following configuration groups. Exact names and defaults must be checked against the current **.env.example** and **app/config.py**.

Configuration group	Examples / purpose
Database	DATABASE_URL — Neon PostgreSQL connection string.

Configuration group	Examples / purpose
LLM	OPENAI_API_KEY and OPENAI_MODEL — model provider credentials and selected model.
Vector database	PINECONE_API_KEY, PINECONE_INDEX_NAME, PINECONE_INDEX_HOST, PINECONE_CLOUD, PINECONE_REGION.
Embeddings	EMBEDDING_PROVIDER, EMBEDDING_MODEL, EMBED_DIM, EMBEDDING_DEVICE, NORMALIZE_EMBEDDINGS.
AWS	AWS_REGION, S3_BUCKET, and credentials or task-role configuration.
RAG behavior	MIN_RELEVANCE, retrieval top_k, and memory turn count.
Application	APP_NAME, ENVIRONMENT, LOG_LEVEL, HITL_ENABLED, MAX_UPLOAD_BYTES.

Never paste secrets into documentation
Use placeholder values in examples. Store actual credentials in environment variables or an approved secrets manager.

Appendix B. Useful SQL queries

Recent audit events

```
SELECT action, actor, tenant_id, detail, created_at
FROM audit_events
ORDER BY created_at DESC
LIMIT 20;
```

Pending approvals

```
SELECT id, tenant_id, requester, question, created_at
FROM approvals
WHERE status = 'pending'
ORDER BY created_at DESC;
```

Document counts by status

```
SELECT status, COUNT(*) AS count
FROM documents
GROUP BY status
ORDER BY status;
```

Recent conversations

```
SELECT tenant_id, conversation_id,
       COUNT(*) AS messages,
       MAX(created_at) AS last_message
FROM chat_messages
GROUP BY tenant_id, conversation_id
ORDER BY last_message DESC
LIMIT 10;
```

Appendix C. Source-content validation notes

The supplied source contains valuable implementation detail, but several statements conflict or require confirmation. These are deliberately surfaced here so the documentation remains trustworthy.

Area	Issue found in source	Action before publication
Development stages	One paragraph says nine stages, while headings enumerate Stage 0 through Stage 11.	Use the current project history or repository milestones to confirm the actual sequence.
Test totals	The summary says 146/146; the listed module counts add to 165.	Run pytest -q and report the current collected/passed/failed counts.
LLM model	Different sections mention gpt-4o-mini, while current project configuration may use another model.	Document the exact OPENAI_MODEL value from the active environment, without exposing the key.
Database tables	Seven tables are listed, but detailed CREATE TABLE examples are supplied for only some of them.	Use infra/schema.sql as the definitive schema reference.
Pinecone region	The source gives different Pinecone region values in separate sections.	Confirm the index host, cloud, and region in Pinecone and app configuration.
Deployment status	A live ALB URL is stated, but a URL in notes does not prove the current service is healthy or deployed.	Check ECS service, target group health, and /health before publishing a live URL.
Performance and cost	Latency and monthly cost figures are presented without a measurement method or dated billing evidence.	Label as estimates or replace with measured benchmark and billing data.
Security	Tenant filters are described, but authentication is explicitly out of scope.	Do not describe the system as production-secure multi-tenant until caller identity and tenant authorization are enforced.
File inventory	The source lists files and approximate line counts that may change over time.	Regenerate the tree from the current repository and remove stale line counts.

Document maintenance

Update this document whenever an API contract, database schema, model configuration, embedding dimension, cloud resource, security control, or deployment procedure changes. Keep example commands safe to copy, use placeholders for account-specific identifiers, and verify all operational claims against the current environment.

End of document.